

13 — CNNs & Training Techniques: Deep Learning in Practice

Time: ~5-6 hours | **Level:** Intermediate → Advanced

What you'll learn:

- CNN architecture evolution: LeNet → ResNet → EfficientNet
- Building residual blocks and mini-ResNet
- Transfer learning: freeze, fine-tune, feature extraction
- Learning rate schedulers: finding the right schedule
- Regularization: dropout, weight decay, label smoothing
- Mixed precision training: 2x faster with no accuracy loss
- Training diagnostics: detecting and fixing common issues

Prerequisites: Notebook 04 (PyTorch Deep Learning), Notebook 11 (Math)

Deep Learning Success = Architecture + Training Tricks

The same architecture can give 60% or 95% accuracy depending on how you train it. This notebook covers the tricks that professionals use.

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import torch
import torch.nn as nn
import torch.nn.functional as F

sns.set_theme(style='whitegrid', font_scale=1.1)
np.random.seed(42)
torch.manual_seed(42)
```

1. CNN Architecture Evolution

Architecture	Year	Key Innovation	Parameters	Top-1 Accuracy
LeNet-5	1998	First practical CNN	60K	—
AlexNet	2012	ReLU, dropout, GPU training	61M	63.3%
VGG-16	2014	Deeper with 3×3 convolutions	138M	74.4%
GoogLeNet	2014	Inception modules (multi-scale)	6.8M	74.8%

Architecture	Year	Key Innovation	Parameters	Top-1 Accuracy
ResNet-50	2015	Skip connections (go deeper!)	25.6M	76.1%
EfficientNet-B0	2019	Compound scaling (width×depth×resolution)	5.3M	77.1%

```
In [ ]: # — Residual Block: the key innovation —————

class ResidualBlock(nn.Module):
    """Basic residual block:  $x + F(x)$  where  $F = \text{conv} \rightarrow \text{BN} \rightarrow \text{relu} \rightarrow \text{conv} \rightarrow \text{BN}$ """
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride=stride,
                                padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        # Skip connection (identity or projection)
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, 1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels),
            )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += self.shortcut(x) # ← The skip connection!
        return F.relu(out)

class MiniResNet(nn.Module):
    """Small ResNet for CIFAR-10."""
    def __init__(self, num_classes=10):
        super().__init__()
        self.prep = nn.Sequential(
            nn.Conv2d(3, 64, 3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(),
        )
        self.layer1 = self._make_layer(64, 64, num_blocks=2, stride=1)
        self.layer2 = self._make_layer(64, 128, num_blocks=2, stride=2)
        self.layer3 = self._make_layer(128, 256, num_blocks=2, stride=2)
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(256, num_classes),
        )

    def _make_layer(self, in_ch, out_ch, num_blocks, stride):
        layers = [ResidualBlock(in_ch, out_ch, stride)]
        for _ in range(1, num_blocks):
            layers.append(ResidualBlock(out_ch, out_ch, 1))
```

```

        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.prep(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        return self.classifier(x)

model = MiniResNet()
dummy = torch.randn(1, 3, 32, 32)
out = model(dummy)
total_params = sum(p.numel() for p in model.parameters())
print(f"MiniResNet output shape: {out.shape}")
print(f"Total parameters: {total_params:,}")

```

2. Transfer Learning: Standing on the Shoulders of Giants

Three strategies:

Strategy	Freeze What	When to Use
Feature extraction	Freeze ALL backbone layers	Very small dataset, similar domain
Fine-tune last layers	Freeze early layers, tune later	Medium dataset
Full fine-tuning	Tune everything (small LR)	Large dataset, different domain

```

In [ ]: # — Transfer learning with pretrained ResNet —————
from torchvision import models

# Load pretrained ResNet-18
resnet = models.resnet18(weights='DEFAULT')

# Strategy 1: Feature extraction (freeze everything, replace head)
for param in resnet.parameters():
    param.requires_grad = False

resnet.fc = nn.Linear(resnet.fc.in_features, 10) # New head for 10 classes

trainable = sum(p.numel() for p in resnet.parameters() if p.requires_grad)
total = sum(p.numel() for p in resnet.parameters())
print(f"Feature extraction:")
print(f"  Trainable: {trainable:,} / {total:,} ({100*trainable/total:.1f}%)")

# Strategy 2: Fine-tune last block + head
resnet2 = models.resnet18(weights='DEFAULT')
for param in resnet2.parameters():
    param.requires_grad = False
# Unfreeze layer4 (last residual block) and fc
for param in resnet2.layer4.parameters():
    param.requires_grad = True
resnet2.fc = nn.Linear(resnet2.fc.in_features, 10)

```

```

trainable2 = sum(p.numel() for p in resnet2.parameters() if p.requires_grad)
print(f"\nFine-tune last block + head:")
print(f"  Trainable: {trainable2:,} / {total:,} ({100*trainable2/total:.1f}%

# Strategy 3: Full fine-tuning
resnet3 = models.resnet18(weights='DEFAULT')
resnet3.fc = nn.Linear(resnet3.fc.in_features, 10)
trainable3 = sum(p.numel() for p in resnet3.parameters() if p.requires_grad)
print(f"\nFull fine-tuning:")
print(f"  Trainable: {trainable3:,} / {total:,} ({100*trainable3/total:.1f}%

```

3. Learning Rate Schedulers

The learning rate is the single most important hyperparameter. A good schedule:

1. **Warm up** slowly (avoid initial instability)
2. **High LR** during middle (explore loss landscape)
3. **Decay** at the end (converge precisely)

```

In [ ]: # — Learning rate schedules comparison —————

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
epochs = 100

# Setup dummy model and optimizer for each scheduler
def get_lrs(scheduler_fn, epochs):
    model = nn.Linear(10, 1)
    optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
    scheduler = scheduler_fn(optimizer)
    lrs = []
    for _ in range(epochs):
        lrs.append(optimizer.param_groups[0]['lr'])
        optimizer.step()
        scheduler.step()
    return lrs

# StepLR
lrs_step = get_lrs(lambda opt: torch.optim.lr_scheduler.StepLR(opt, step_size=30, gamma=0.1), epochs)
axes[0, 0].plot(lrs_step, 'b-', linewidth=2)
axes[0, 0].set_title('StepLR (drop every 30 epochs)')
axes[0, 0].set_ylabel('Learning Rate')

# CosineAnnealing
lrs_cosine = get_lrs(lambda opt: torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs), epochs)
axes[0, 1].plot(lrs_cosine, 'r-', linewidth=2)
axes[0, 1].set_title('CosineAnnealingLR')

# OneCycleLR
def get_onecycle_lrs(epochs):
    model = nn.Linear(10, 1)
    optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
    scheduler = torch.optim.lr_scheduler.OneCycleLR(optimizer, max_lr=0.1, t

```

```

lrs = []
for _ in range(epochs):
    lrs.append(optimizer.param_groups[0]['lr'])
    optimizer.step()
    scheduler.step()
return lrs

lrs_onecycle = get_onecycle_lrs(epochs)
axes[1, 0].plot(lrs_onecycle, 'g-', linewidth=2)
axes[1, 0].set_title('OneCycleLR (warm up + cosine decay)')
axes[1, 0].set_xlabel('Epoch')
axes[1, 0].set_ylabel('Learning Rate')

# Warmup + cosine (manual)
def warmup_cosine(epoch, warmup=10, max_epochs=100, base_lr=0.1):
    if epoch < warmup:
        return base_lr * epoch / warmup
    progress = (epoch - warmup) / (max_epochs - warmup)
    return base_lr * 0.5 * (1 + np.cos(np.pi * progress))

lrs_warmup = [warmup_cosine(e) for e in range(epochs)]
axes[1, 1].plot(lrs_warmup, 'm-', linewidth=2)
axes[1, 1].set_title('Warmup + Cosine (custom)')
axes[1, 1].set_xlabel('Epoch')

plt.suptitle('Learning Rate Schedules', fontsize=14)
plt.tight_layout()
plt.show()

print("Rule of thumb:")
print(" - OneCycleLR for fast training (super-convergence)")
print(" - CosineAnnealing for fine-tuning pretrained models")
print(" - Warmup + cosine for training large models from scratch")

```

4. Regularization Techniques

Technique	Mechanism	Typical Use
Dropout	Randomly zero activations (training only)	FC layers, attention
Weight decay	Add $\lambda w ^2$ to loss	Always (usually 0.01)
Label smoothing	Soft targets: 0.9/0.1 instead of 1/0	Classification
Data augmentation	Transform training images	Image models
Early stopping	Stop when val loss stops improving	All models

```

In [ ]: # — Label smoothing implementation —————

class LabelSmoothingCE(nn.Module):
    """Cross-entropy with label smoothing."""
    def __init__(self, smoothing=0.1):
        super().__init__()
        self.smoothing = smoothing

```

```

def forward(self, pred, target):
    n_classes = pred.size(-1)
    log_probs = F.log_softmax(pred, dim=-1)

    # One-hot with smoothing
    with torch.no_grad():
        smooth_target = torch.full_like(log_probs, self.smoothing / (n_classes + 1))
        smooth_target.scatter_(1, target.unsqueeze(1), 1.0 - self.smoothing)

    loss = (-smooth_target * log_probs).sum(dim=-1).mean()
    return loss

# Compare
logits = torch.randn(4, 10) # 4 samples, 10 classes
targets = torch.tensor([3, 7, 1, 9])

criterion_hard = nn.CrossEntropyLoss()
criterion_smooth = LabelSmoothingCE(smoothing=0.1)

print(f"Hard labels CE loss: {criterion_hard(logits, targets):.4f}")
print(f"Smooth labels CE loss: {criterion_smooth(logits, targets):.4f}")
print(f"\nLabel smoothing prevents the model from being overconfident")
print(f"Instead of targeting [0, 0, 0, 1, 0, ...], it targets [0.011, 0.011, ...]")

```

5. Mixed Precision Training

Use FP16 for forward/backward pass, FP32 for parameter updates. Result: ~2x faster, ~0.5x memory, same accuracy.

```

# The modern way (PyTorch 2.0+)
scaler = torch.amp.GradScaler()

for batch in dataloader:
    optimizer.zero_grad()

    with torch.amp.autocast(device_type='cuda',
dtype=torch.float16):
        output = model(batch)
        loss = criterion(output, targets)

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()

```

```

In [ ]: # — Training diagnostics: what to watch —————

# Simulate training metrics
np.random.seed(42)
epochs = range(1, 51)

# Good training
train_loss_good = 2.0 * np.exp(-np.array(epochs)/10) + 0.3 + np.random.normal(

```

```

val_loss_good = 2.0 * np.exp(-np.array(epochs)/12) + 0.35 + np.random.normal(
# Overfitting
train_loss_over = 2.0 * np.exp(-np.array(epochs)/5) + 0.1 + np.random.normal
val_loss_over = 2.0 * np.exp(-np.array(epochs)/15) + 0.5 + 0.01 * np.array(e

# Underfitting
train_loss_under = 2.0 * np.exp(-np.array(epochs)/30) + 1.0 + np.random.normal
val_loss_under = 2.0 * np.exp(-np.array(epochs)/30) + 1.05 + np.random.normal

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, (tl, vl, title) in zip(axes, [
    (train_loss_good, val_loss_good, 'Good Training'),
    (train_loss_over, val_loss_over, 'Overfitting (↑ regularization)'),
    (train_loss_under, val_loss_under, 'Underfitting (↑ capacity)'),
]):
    ax.plot(epochs, tl, 'b-', label='Train loss', linewidth=2)
    ax.plot(epochs, vl, 'r-', label='Val loss', linewidth=2)
    ax.fill_between(epochs, tl, vl, alpha=0.1, color='orange')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.set_title(title)
    ax.legend()

plt.suptitle('Training Diagnostics: Read Your Loss Curves', fontsize=14)
plt.tight_layout()
plt.show()

```

Key Takeaways

Concept	One-Line Summary
ResNet	Skip connections solve vanishing gradients → train 100+ layer networks
Transfer learning	Freeze backbone → fine-tune head → unfreeze last layers
OneCycleLR	Warmup + high LR + cosine decay = fast convergence
Label smoothing	Prevents overconfidence; use smoothing=0.1 for classification
Mixed precision	FP16 forward/backward + FP32 updates = 2x faster, same accuracy
Training diagnostics	Train-val gap = overfitting; both high = underfitting

What to study next:

- **Notebook 14:** GenAI, Embeddings and RAG
- **Notebook 17:** MLOps and experiment tracking